



Git - система контроля версий

[Основной источник](#)

Содержание

- [Инициализация](#)
- [Branch](#)
- [Staging_area](#)
- [Commit](#)
- [Stash](#)
- [.gitignore](#)
- [Merge](#)
- [GitHub](#)

Инициализация

Вывести конфигурацию

```
git config --list
```

Задать информацию

```
git config --global user.name bigboss
```

`--global` - для всех репозиториев

Если имя больше одного слова, то нужно заключать в "кавычки"

Создать новый репозиторий или реинициализировать старый

```
git init
```

Подробнее о начале работы в разделе [GitHub](#)

Branch

- указатель на последний коммит в ветке.

Слова **master** (хозяин) и **slave** (раб) имеют глубокие исторические корни, связанные с рабством и угнетением. **Github** называет главную ветку `main`, а **Git** - `master`.

Нам же следует называть ее `main` дабы не угнетать угнетенных и работать с гитхабом.

Переименовать ветку

```
git branch -m main
```

Создать ветку

```
git branch newbranch
```

Удалить ветку

```
git branch -d oldbranch
```

Удалить текущую ветку нельзя. Возможна ошибка, для принудительного удаления `git branch -D oldbranch`

Просмотреть список веток

```
git branch -a
```

`git branch` - выводит текущую ветку

Переключиться на другую ветку или коммит

```
git checkout main
```

С помощью этой команды мы переключаемся на ветку, что является последним коммитом. Соответственно вместо имени ветки можно указать хэш коммита и перейти на него.

Переключиться на прошлую ветку

```
git checkout -
```

Сомнительно, но окей

Рекомендуется использовать `switch` т.к. он новее, безопаснее и предназначен исключительно для перемещения между ветками

Переключиться на другую ветку

```
git switch main
```

Создать ветку и сразу переключиться на нее

```
git checkout -b newbranch
```

-c - create

Staging area

- область файлов, которые мы готовим к коммиту. Так сказать отслеживаем. Версии файлов хранятся в качестве объектов в скрытой папке `.git`

Статусы файлов:

- **Untracked** - не отслеживаемый
- **Staged** - отслеживаемый
- **Unmodified** - состояние в репозитории и рабочей директорией одинаково
- **Modified** - файл изменен с последнего коммита

Просмотреть staging area

```
git status
```

Добавить файлы в staging area

```
git add filename
```

Можно написать `git add .` чтобы добавить все не отслеживаемые файлы в стэйджинг эрэа.

Перестать отслеживать файл

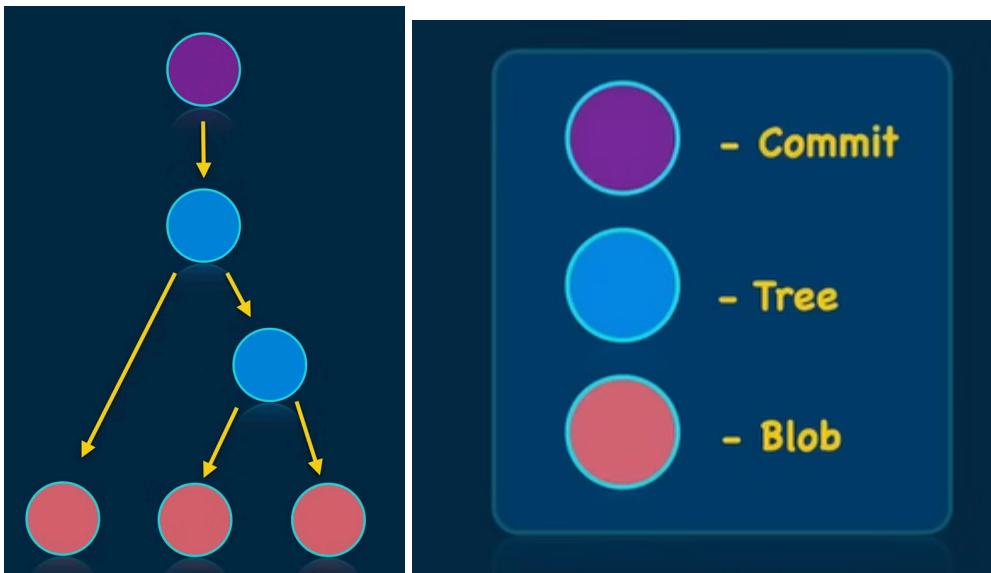
```
git reset filename
```

Commit

- указатель на дерево (версию проекта)

Создать коммит

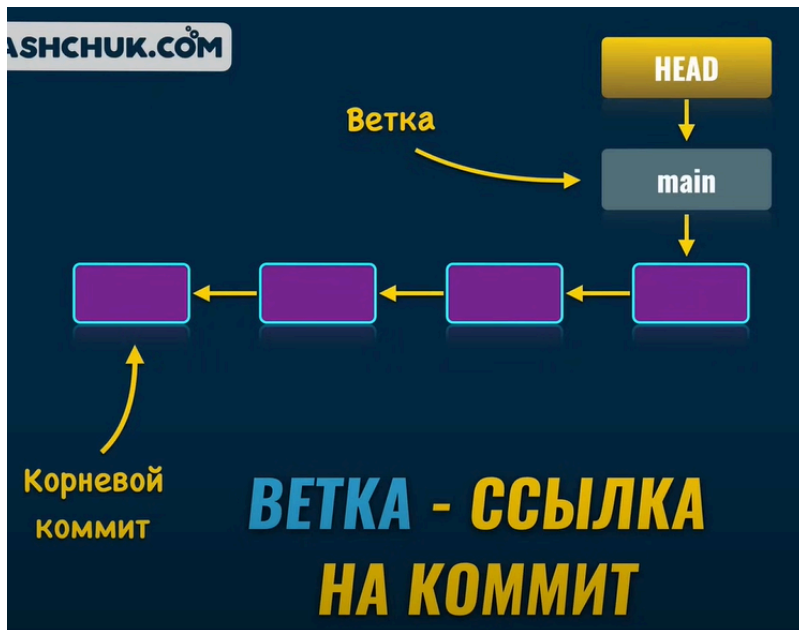
```
git commit -m "message"
```



- blob - файл
- tree - папка

Head - указатель на ветку, которая является указателем на последний коммит в ней или же непосредственно на коммит.

Head detached at "hash" - значит, что сейчас Head ссылается не на ветку (последний коммит), а на коммит из истории. В таком состоянии работать не следует.



Вывести историю коммитов в ветке

```
git log
```

Посмотреть разницу между коммитами

```
git diff main 0f45dfhs
```

`git diff` - выведет разницу между текущим состоянием и последним коммитом

Посмотреть разницу определенного файла

```
git diff index.html
```

Для этого нужно находиться в директории файла

Stash

- временное хранилище

Если нужно переключиться на другую ветку, но не хочется коммитить текущие изменения можно поместить их в stash.

Команды для stash

- `git stash` - временно сохранить текущие изменения
- `git stash push -m "message"` - дописать сообщение

- `git stash list` - просмотреть список stash
- `git stash pop` - вернуть и удалить последние изменения
- `git stash apply` - вернуть изменения без удаления
- `git stash drop` - удалить сохраненные изменения
- `git stash clear` - очистить список stash

Пример

```
git stash list

- stash@{0}: WIP on main: 1234567 Commit message
- stash@{1}: On feature-branch: 7654321 Another commit message

git stash apply stash@{1}

git stash drop stash@{0}
```

.gitignore

Файл `.gitignore` используется в Git для указания файлов и папок, которые не должны отслеживаться и добавляться в репозиторий. Должен располагаться в корневой папке проекта.

Сам файл `.gitignore` должен находиться в **staging area**.

```
# Игнорировать файл
__pychace__/

# Игнорировать папку
folder/

# Игнорировать все файлы с расширением .log
*.log

# Игнорировать всё в папке, кроме конкретного файла
temp/*
!temp/important.txt
```

Основные правила:

- Каждая строка — отдельное правило.
- `#` — комментарий.
- `*` — любое количество символов.

- ? — один любой символ.
- / в начале — указывает на корневую директорию.
- ! — исключение из правил (например, !file.txt).

Merge

При слиянии веток получившийся коммит в `git log` будет иметь историю коммитов обеих веток. Такой коммит называется **Merge commit** и имеет два родительских коммита.

- `feature branch` - ветка, которую сливают в **receiving branch**
- `receiving branch` - ветка, принимающая изменения из **feature branch**

Совместить ветки

```
git merge feature_branch
```

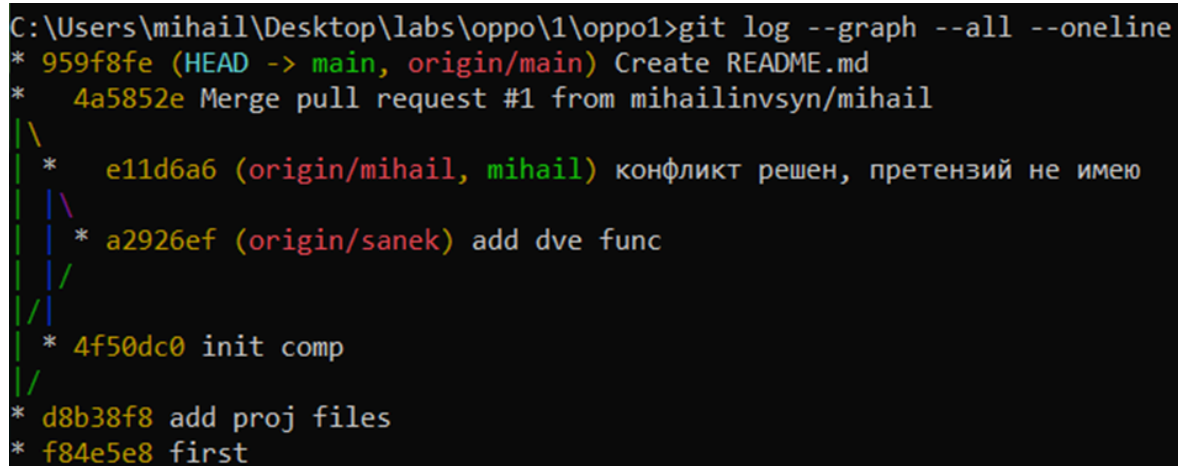
Для этого нужно находиться в принимающей ветке **receiving branch**.

Желательно писать message

```
git merge -m "merging branch1 into main" branch1
```

Отобразить дерево веток

```
git log --graph --all --oneline
```



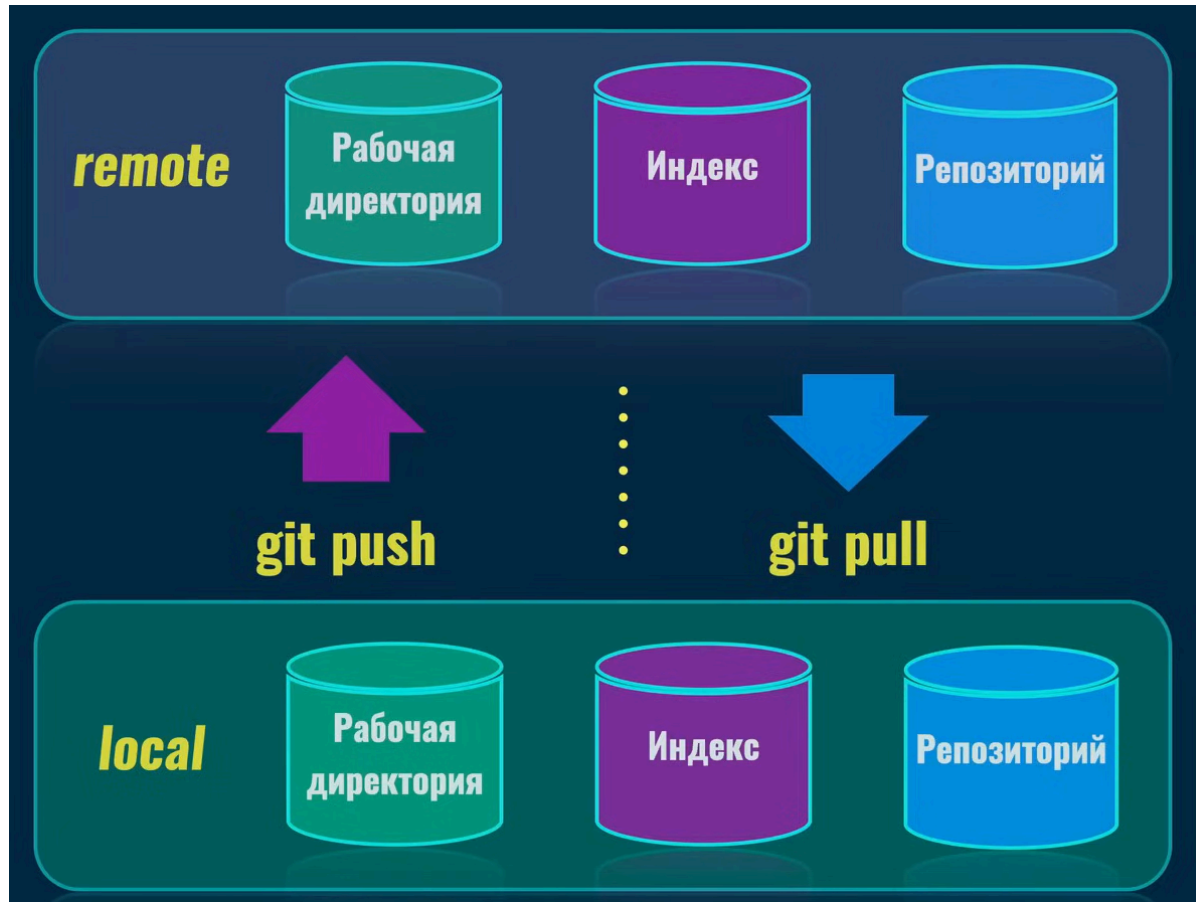
```
C:\Users\mi hail\Desktop\labs\oppo\1\oppo1>git log --graph --all --oneline
* 959f8fe (HEAD -> main, origin/main) Create README.md
* 4a5852e Merge pull request #1 from mi hailinvsyn/mi hail
|
| * e11d6a6 (origin/mi hail, mi hail) конфликт решен, претензий не имею
| |
| | * a2926ef (origin/sanek) add dve func
| |
| |
| |
| * 4f50dc0 init comp
|
|
* d8b38f8 add proj files
* f84e5e8 first
```

Сбросить ветку до состояния другой ветки

```
git reset --hard main
```

Эта команда текущую ветку точной копией ветки `main`. **Важно** : все изменения в ветке `mihaail`, которые не были слиты в `main`, будут потеряны.

GitHub



Клонировать репозиторий локально

```
git clone <url>
```

- `<url>` - ссылка на репозиторий.
Клонирование можно сделать по протоколам `https` и `ssh`.
Публичный репозиторий может клонировать любой пользователь.

Origin - имя удаленного репозитория по умолчанию

(с английского источник, начало)

Загрузить изменения с удаленной ветки

```
git pull
```


Отправить изменения в удаленную ветку

```
git push
```

Вывести связь между локальным и удаленным репозиторием

```
git remote -v
```

Вывести связь между локальной и удаленной веткой

```
git remote -vv
```

Добавить удаленный репозиторий

```
git remote add https://...repo.git
```

Удалить удаленный репозиторий

```
git remote remove origin
```

Если локально что-то есть и хочется в гитхаб

- Создаем репозиторий локально `git init`
- Делаем коммит `git commit -m "first commit"`
- Переименовываем *master* в *main* `git branch -M main`
- Подключаем удаленный репозиторий `git remote add origin https://... origin` - общепринятое название удаленного сервера. Можно назвать иначе, но не нужно.
- Пушим имеющиеся коммиты на сервер `git push -u origin main`

Флаг `-u`, тоже самое, что и `--set-upstream`, нужно писать только при первом пуше чтобы связать локальную ветку с удаленной. После можно писать просто `git push`

Если друзья уже все сделали и нужно подключиться

- Владелец репозитория должен добавить вас в список *collaborators*
- Нужно клонировать репозиторий `git clone <url>`
- Создаем локальную ветку `git checkout -b my_branch`
- После создания нужных коммитов, пушим локальную ветку в удаленную `git push -u origin my_remote_branch`

1. Флаг `-u` нужно писать только в первый раз для связи локальной и удаленной ветки.
2. Если ветки `my_remote_branch` не существует, то github создаст ее.
3. Локальную и удаленную ветку следует называть одинаково.
4. Дальше изменения можно пушить командой `git push`.

Собственность

Если ваш друг создал репозиторий, он может дать вам права на чтение и/или запись в репозиторий. Если вы внесли вклад в репозиторий, то есть пушили что-то в `main`, то обретаете статус **"contributor"**. Если же хочется, чтобы репозиторий также был в списке **"My repositories"**, нужно, чтобы проект принадлежал не другу, а *организации*. Тогда он может дать вам права **owner** и вы будете включены как член организации.